Reviewer Pack — Free Cumulant Gap in Read-Twice BPs Bounds Disjointness Comm…

Entry 6004bf0dc5da · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 07:12:31 UTC

Free Cumulant Gap in Read-Twice BPs Bounds Disjointness Communication Complexity

Entry ID: `6004bf0dc5da` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 07:12:31 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Disjointness Communication Complexity

Statement:

> For a random 3-CNF formula Φ on n variables, let $\mu(\Phi)$ denote the free cumulant gap between its clause indicator variables. Then the randomized communication complexity $R_{1/3}(DISJ\_n) \geq \Omega(\mu(\Phi) / \log n)$.

Rationale (proposer's reasoning):

> Free cumulants capture non-crossing partition dependencies in clause variables, which may reveal structural asymmetries in disjointness protocols. The gap between cumulant growth rates in read-twice BPs and the exponential separation in communication complexity could expose hidden algebraic constraints in parity-like functions.

Taxonomy category: `BP_READTWICE` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6d36a794d6dbc098`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
            A[i], A[max_row] = A[max_row], A[i]
            for j in range(i+1, n):
                factor = A[j][i] / A[i][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
        return A

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def moment_cumulant_transform(A):
        n = len(A)
        cumulants = [0] * (n + 1)
        cumulants[0] = 1
        for i in range(n):
            for j in range(i, n):
                factor = A[i][j]
                for k in range(j+1, n):
                    factor *= A[j][k]
                cumulants[i+1] += factor
        return cumulants

    def free_cumulant_gap(cumulants):
```

```
        gap = 0
        for i in range(2, len(cumulants)):
            gap += (cumulants[i] - sum(cumulants[:i])) / (i * math.log(i))
        return gap

    n = 40
    num_clauses = random.randint(n, 3*n)
    clauses = []
    for _ in range(num_clauses):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
        clauses.append(clause)

    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(i, n):
            count = sum(1 for clause in clauses if (i+1 in clause and j+1 in clause) or (-i-1 in c
            A[i][j] = A[j][i] = count

    A = gaussian_elimination(A)
    cumulants = moment_cumulant_transform(A)
    mu = free_cumulant_gap(cumulants)

    R_disj = 2 * n * math.log(n) / (math.log(2) ** 2)

    return {
        "metric_name": "mu/R_disj",
        "metric_value": mu / R_disj,
        "instances_tested": 1,
        "conjecture_holds": mu / R_disj >= 0.1,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1506b8e3.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1506b8e3.py", line 72, in run_trial
    A = gaussian_elimination(A)
        ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1506b8e3.py", line 27, in gaussian_elim
    factor = A[j][i] / A[i][i]
             ~~~~~~~~^~~~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with ZeroDivisionError, preventing data collection. The pre-registered support condition cannot be evaluated without successful trials. | next: Debug gaussian_elimination() to handle zero denominators, then re-run trials with diverse seeds

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	32454
2	preregistration	ollama_remote	qwen3:8b	0	0	19942
3	novelty	ollama_remote	qwen3:8b	0	0	16543
4	novelty	ollama_remote	qwen3:8b	0	0	10642
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9541
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10465
7	judge	ollama_remote	qwen3:8b	0	0	11773

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 111360 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/6004bf0dc5da.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6004bf0dc5da.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6004bf0dc5da.tar.gz` (if generated)